15.10
Microservices Anti-patterns & Migration

What goes wrong and how to fix it

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

Common Anti-patterns

Not all 'microservices' are truly decoupled. These are the most common traps teams fall into — and how to recognize them.

Distributed Monolith	Services deploy independently but share a database or have tight sync HTTP coupling everywhere	Give each service its own DB. Use events for cross-service communication
Chatty Services	API gateway calls 6 services to render one page — slow, fragile	Use Backend-for-Frontend (BFF) or GraphQL aggregation layer
Too-Fine Grained	100 services for a 5-person team — unmanageable operational overhead	Merge services. Rule of thumb: one team, one service (or 2-3 max)
Shared Libraries	All services import the same 'common' library — creates hidden coupling	Libraries for utilities only, never for domain models. Duplicate the model per service

Migration Strategy: Strangler Fig Pattern

The safest way to migrate a monolith to microservices is the Strangler Fig: route all traffic through an API Gateway, then extract services one route at a time. The monolith 'shrinks' while new services grow alongside it.

■ **Simple Analogy:** A strangler fig tree grows around an old tree, slowly replacing it. By the time the old tree is gone, the new tree is fully established. No big-bang rewrite needed.

Phase	What Happens
Phase 1	Put an API Gateway in front of the monolith. All traffic still goes to monolith.
Phase 2	Extract first service (e.g., Payments). Gateway routes /payments to new svc, rest to monolith.
Phase 3	Extract second service (e.g., Inventory). Monolith handles fewer routes.
Phase N	Monolith is empty. All traffic served by microservices. Decommission monolith.

Dual-Write Data Migration

When migrating a service's data from the monolith's DB to its own DB:

1. Write to BOTH old DB and new DB simultaneously (dual-write)

2. Compare reads: shadow-read from new DB and compare with old — log any discrepancies
3. Once confident: switch reads to new DB
4. Remove writes to old DB
5. Decommission old table

Key Lesson: Start with the least risky service to extract. Build confidence and tooling before extracting core services. A failed microservices migration is usually caused by extracting too much too fast.

■ **Real-World Example:** eBay: Had a shared Oracle database across all 'microservices' — a classic distributed monolith. Fix: API Gateway in front, then extracted Search, Recommendations, Payments one by one over 3 years. Zalando used dual-write for 6 months when migrating their Catalog from PostgreSQL to MongoDB.